

# SunoBolo — Voice-Based Inventory Management Using Conversational Speech

## Milestone 1.4 · UI/UX Design & Prototype

**Deliverable:** Clickable Prototype / UI Design

**Date:** 19 September 2026

**Prepared by:** \_\_\_\_\_

---

### Project Context

**SunoBolo** is a voice-first inventory system for small Indian retail shops.

Every voice inventory product on the market works as a **command interface**: you press a button and issue a structured instruction — *"add twenty kilo rice."* That is still data entry, performed with your mouth instead of your thumbs. SunoBolo inverts this. The phone sits on the shop counter and **listens to the shop**, extracting stock movements from the ordinary conversation between owner and customer — sentences that were never addressed to the app at all. A kirana shop already says every transaction out loud, usually twice: once when the customer asks and once when the owner confirms. That speech is a complete, free transaction log that currently evaporates into the air. **SunoBolo captures it.**

Because passive capture is less accurate than dictated commands, nothing it extracts is written to stock automatically. Every extraction becomes a **proposal** the owner approves in bulk, and a conventional **Command Mode** mic button remains available as the reliable path. The novel capability is genuinely novel; the failure mode is an ordinary voice inventory app.

### Milestone 1 deliverables

- **1.1** — Requirements Gathering & Analysis
- **1.2** — Functional Requirements
- **1.3** — Technical Design & Architecture
- **1.4** — UI/UX Design & Prototype (*this document*)

---

---

## UI/UX Design & Prototype

### Design Principles

1. **Nothing to operate in the happy path.** Counter Mode is a toggle, not a workflow. The owner's hands stay on the goods.
2. **Attention is the scarce resource.** The app interrupts only when it has something worth approving, and never shows a low-confidence guess.
3. **Every correction is a tap.** The keyboard is a failure state, not a feature.
4. **Colour, icon and word together.** Never colour alone — users may be reading only the numerals.
5. **Trade language only.** If a shopkeeper wouldn't say it across the counter, it doesn't appear on screen.

### Screens

**A • Login.** Shop name, language picker (हिंदी / తెలుగు / English), 4-digit PIN pad.

Large keys, no text entry anywhere.

**B • Dashboard (home).** Shop header with language picker. Low-stock banner when anything needs attention. Search field matching any alias in any language. Item rows sorted needs-attention-first, each showing the item name in the chosen language with its English name beneath, the current quantity in its display unit, and a colour-coded status pill (green healthy / amber low / red out). A bottom bar carries Stock, the Counter Mode toggle, and Alerts with a live proposal count.

**C • Counter Mode (active).** A persistent, unmistakable listening indicator — this is a privacy commitment rendered as UI. Live transcript text scrolls as words are recognised. Proposals appear as cards as they are extracted, without interrupting anything.

**D • Pending tray.** Proposal cards, newest first. Each shows item, quantity, unit,

direction, confidence badge, and the sentence it came from in quotes — so the owner can see *why* the app thinks what it thinks. Steppers for quantity, chips for unit and direction, "did you mean" chips when confidence is middling. One **Accept all** control at the top for everything above threshold.

**E • Command Mode sheet.** Pulsing mic while listening, live transcript, then a confirmation card identical in layout to a proposal card — the same vocabulary of controls, learned once.

**F • Item detail.** Large current quantity with status. Pack-size facts (*1 bori = 50 kg*), reorder level, last price. Recent history with each entry's original sentence in quotes and its signed effect on stock.

**G • Alerts.** Low and out-of-stock items with reorder suggestions in **purchase** units — "*order 2 bori (108 kg)*" — because that is the unit the owner will actually buy in.

## Visual Language

- Deep teal primary (#0f766e) — calm, trustworthy, distinct from the red/orange used by every billing app
- Status: green #16a34a healthy · amber #d97706 low · red #dc2626 out
- System font stack including Noto Sans Devanagari and Noto Sans Telugu
- 16 px base type, 22 px quantities, 48 px minimum touch targets
- Card-based layout on a warm off-white background, one idea per card

## Prototype

A clickable mobile-first prototype has been built as the **\*\*real** frontend shell of the application\*\*, not a throwaway mockup: the same HTML file becomes the production UI in Milestone 2. Screens A, B, D, E, F and G are implemented and navigable, with the speech layer live and the data layer stubbed pending the Milestone 2 backend.

*Prototype link: [\\_\(deployed URL — add before submitting\)\\_](#)*

*Source repository: [\\_\(GitHub URL — add before submitting\)\\_](#)*

---

## Appendix — Honest Assessment of Risk

The passive-capture approach is deliberately more ambitious than command-mode voice entry,

and it is weaker in one specific respect: \*\*extraction accuracy from unstructured conversation will be materially lower than from deliberate commands\*\* — realistically 50–65% against roughly 90%.

This is designed around rather than hidden:

- Counter Mode **proposes**; it never commits. A wrong extraction costs one dismissive tap.
- Below-threshold extractions are discarded rather than shown, so the tray stays trustworthy.
- **Command Mode remains available at all times** as the reliable path. The passive feature is

the differentiator; the command path is the floor the product cannot fall through.

The result is a system whose novel capability is genuinely novel, and whose failure mode is an ordinary, well-understood voice inventory app.